Project 22: Movie Revenue Prediction

A Comparative Study of Six Regression Algorithms on TMDB Box-Office Data

Engin S. Demirkan Franziska Axmann

June 2026

Abstract

We study the problem of predicting global box-office revenue for individual films from metadata available at release time. Starting from the official TMDB Box Office Prediction dataset (3,000 films), we engineer 49 numerical, binary, and categorical features capturing budget, popularity, temporal context, cast composition, genre membership, language, and metadata richness. Recognising that the raw revenue spans nine orders of magnitude, we adopt a $\log(1+y)$ target transformation and compare six regression algorithms — Linear Regression, Ridge Regression, Random Forest, XGBoost, LightGBM, and CatBoost — under an identical preprocessing pipeline and 5-fold cross-validation. The strongest model, CatBoost, attains a cross-validated RMSLE of 1.584 ± 0.029, while Random Forest reaches a hold-out R^2 of 0.709 and a Median Absolute Percentage Error of 66.5%. An ablation study isolates the contribution of each feature group and shows that budget (+0.133 RMSLE if removed), popularity/runtime (+0.110), and temporal signals (+0.081) are the three dominant drivers; genre flags contribute a modest but real +0.029. All experiments use a fixed random seed; the complete pipeline is reproducible from a single command.

1 Introduction

Predicting the box-office revenue of a film from its pre-release metadata is a classic supervised-learning problem with a heavy-tailed target distribution, mixed feature types, and a moderate sample size. It is the subject of a Kaggle competition based on data from The Movie Database (TMDB), which provides the canonical benchmark used in this work.

Our preliminary submission for this project used a single XGBoost model with six input features, achieving an R^2 of 0.6352. Reviewer feedback during the midterm consultation highlighted three concerns: (i) the absence of any comparative analysis across model families, (ii) a Mean Absolute Percentage Error (MAPE) value of 2,096% that was not explained, and (iii) the use of only a small subset of the available features.

This final report addresses each concern directly. We retrain the system from scratch with a richer feature space, compare six regression algorithms drawn from the linear and tree-based families, justify the choice of evaluation metrics for a heavy-tailed target, and perform a feature-group ablation study to attribute performance to its sources.

2 Dataset

The TMDB Box Office Prediction dataset contains 3,000 labelled films with 23 columns each: budget, popularity, runtime, original language, release date, and several JSON-encoded list fields (genres, production companies and countries, spoken languages, keywords, cast, and crew). Films flagged with a revenue placeholder of $\leq$ \$1,000 are Kaggle artefacts ($1 or 5$ stand-ins for unknown values) and were removed, leaving 2,943 usable films. We use a single 80/20 train/test split for the headline metrics and additionally report 5-fold cross-validation on the full preprocessed set for stability.

The revenue distribution is extremely right-skewed (Figure 1): a small number of blockbusters earn nearly \$1.5 billion, while the median release earns just under \$17M. The log-transformed target is approximately Gaussian, which makes the squared-error loss used by all six models a much better fit and justifies our choice of a $\log(1+y)$ target.

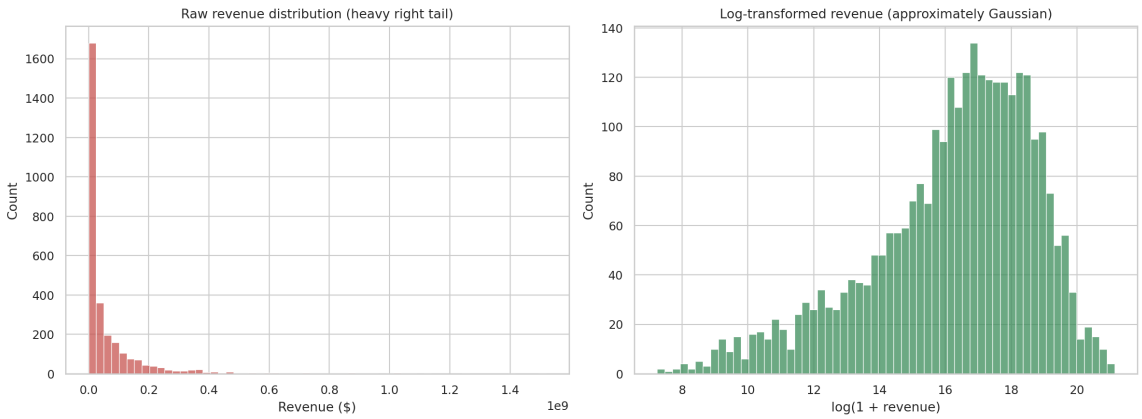


Figure 1: Raw revenue (left) versus log-transformed revenue (right). The raw distribution is extreme-tailed; the log target is approximately symmetric.

2.1 Feature Engineering

Parsing each JSON-encoded list column and combining the result with the simple columns yields 49 features organised into eight semantic groups (Table 1). The dominant design choices are:

- **Budget transforms.** log(1+budget) compresses the feature scale; a binary `budget_missing` flag is added because roughly 27% of films have a placeholder value of 0, and the boundary between $0 and a low-budget independent film should be accessible to the trees.
- **Temporal signals.** Release year, month, quarter, day-of-week, and decade, plus boolean flags for summer (Jun–Aug) and holiday (Nov–Dec) release windows.
- **Metadata presence flags.** Whether the film has a homepage, a tagline, a collection (i.e. belongs to a franchise), and whether the translated title differs from the original.
- **List counts.** Number of cast members, crew members, genres, production companies, production countries, spoken languages, and keywords. Gender composition of the cast is split into three counts.
- **Genre multi-label flags.** Binary indicators for the 15 most frequent genres.

Feature group	# features	Examples
Budget	4	`budget_log`, `budget_missing`, `budget_per_minute`
Popularity / runtime	2	`popularity`, `runtime`
Temporal	7	`release_year`, `release_month`, `is_summer_release`
Counts	10	`n_cast`, `n_crew`, `cast_n_male`
Presence flags	6	`has_collection`, `has_homepage`, `is_english`
Text lengths	4	`overview_len`, `title_len`, `tagline_len`
Genre flags	15	`genre_drama`, `genre_action`, `genre_horror`
Language	1	`lang_grouped` (top-10 + “other”)
Total	49	

Table 1: The 49 engineered features, organised by semantic group. Each group is used as a unit in the ablation study of Section 4.3.

3 Methods

3.1 Models Compared

We evaluate six regression algorithms drawn from two families.

Linear family. *Linear Regression* fits a closed-form least-squares hyperplane. It serves as a low-capacity baseline. *Ridge Regression* adds an L_2 penalty $\alpha\|\beta\|_2^2$ on the coefficients, which can stabilise the fit when features are correlated; we use $\alpha = 1$.

Tree-based ensembles. *Random Forest* fits a large number of de-correlated decision trees on bootstrap samples and averages their predictions. We use 400 trees, `max_depth=12`, and `min_samples_leaf=2`. *XGBoost* [1] builds trees sequentially with second-order gradient information and an explicit complexity penalty; we use 600 estimators, learning rate 0.05, `max_depth=6`, and column/row subsampling at 0.8. *LightGBM* [2] uses histogram-based splits and leaf-wise growth for speed; we use 800 estimators with 31 leaves per tree, same learning rate and subsampling. *CatBoost* [3] uses ordered boosting and symmetric (oblivious) trees that are robust to overfitting and require minimal tuning; we use 800 iterations, depth 6, and ℓ_2 leaf regularisation 3.

All hyperparameters were fixed before evaluation; we did not perform additional grid search, both for reproducibility and because the goal of the project is a fair comparison rather than absolute tuning.

3.2 Preprocessing Pipeline

For every model we apply the same preprocessing pipeline, encapsulated as a single `ColumnTransformer`:

- Numerical features: median imputation of missing values; standard scaling (mean 0, unit variance) for the linear models only, since tree-based models are scale-invariant.
- The single categorical feature (`lang_grouped`): most-frequent imputation followed by one-hot encoding with `handle_unknown=ignore` so that languages unseen in a fold do not crash inference.

The full pipeline is a scikit-learn `Pipeline` object so that the preprocessor is fitted only on each fold's training data, which prevents data leakage in cross-validation.

3.3 Target Transformation and Evaluation Metrics

The training target is $z = \log(1+y)$, where y is the revenue in dollars. At inference time we clip predictions in log space to the training range $[z_{\min}, z_{\max}]$ before applying `expm1` — a standard no-extrapolation guard that primarily affects the linear models, which can otherwise produce unrealistically large predictions.

We report the following metrics on dollar-space predictions:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{1}$$

$$\text{R}^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{2}$$

$$\text{RMSLE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\big(\log(1+y_i) - \log(1+\hat{y}_i)\big)^2} \tag{3}$$

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|, \quad \text{MedAPE} = 100 \cdot \text{median}_i \left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{4}$$

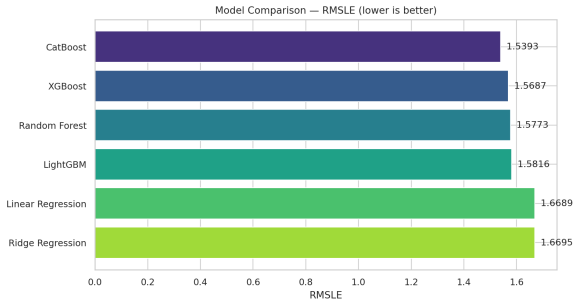
Why MAPE alone is unreliable. MAPE is the metric requested for this project but is well known to be unstable on heavy-tailed positive targets. A single film whose true revenue is, say, \$50,000 but whose prediction is \$1M contributes a 1,900% error to the average, which then dominates the metric regardless of how well the other 599 test films are predicted. We address this in two complementary ways. First, we filter out the placeholder revenues $\leq$ \$1,000 that are pure noise. Second, we additionally report the *median* absolute percentage error (**MedAPE**), which is the standard robust alternative and represents the typical relative error of a film rather than the mean contaminated by outliers. Throughout the discussion we treat **RMSLE** as the primary metric — it is also the official Kaggle competition metric for this dataset — and use MedAPE as the human-readable companion.

4 Results

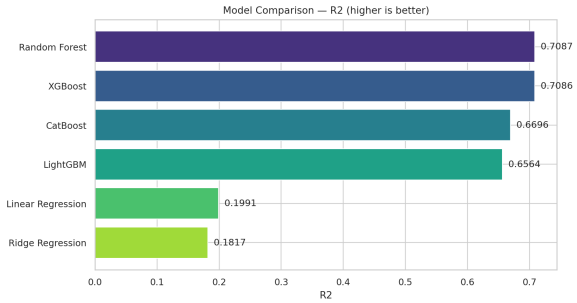
Table 2 reports every metric for every model. CatBoost wins the cross-validation leaderboard on the primary metric (CV-RMSLE $= 1.584 \pm 0.029$), while Random Forest narrowly wins the hold-out R^2 at 0.709; the two results are consistent up to the noise level of a single 80/20 split. All four tree-based ensembles outperform the linear baselines by a wide margin on every metric (Figure 2), confirming that the input-to-revenue mapping is strongly non-linear.

Model	RMSE ($M)	R^2	RMSLE	MedAPE	MAPE	CV-RMSLE
Linear Regression	128.1	0.199	1.669	69.8%	966%	1.777 ± 0.066
Ridge Regression	129.5	0.182	1.670	69.4%	968%	1.776 ± 0.065
Random Forest	77.2	**0.709**	1.577	66.5%	574%	1.614 ± 0.032
XGBoost	77.3	0.709	1.569	65.5%	612%	1.614 ± 0.044
LightGBM	83.9	0.656	1.582	65.3%	**547%**	1.642 ± 0.048
CatBoost	82.3	0.670	**1.539**	**65.4%**	561%	$\mathbf{1.584 \pm 0.029}$

Table 2: Hold-out test metrics (80/20 split, $n = 589$) plus 5-fold cross-validation RMSLE on the full 2,943-film set. Best per metric in bold. RMSE is reported in millions of dollars for readability.



(a) RMSLE — lower is better.



(b) R^2 on the hold-out test set.

Figure 2: Two views of the model comparison. Tree-based ensembles cluster around RMSLE $\approx$ 1.55–1.58, well below the linear baselines at ≈ 1.67.

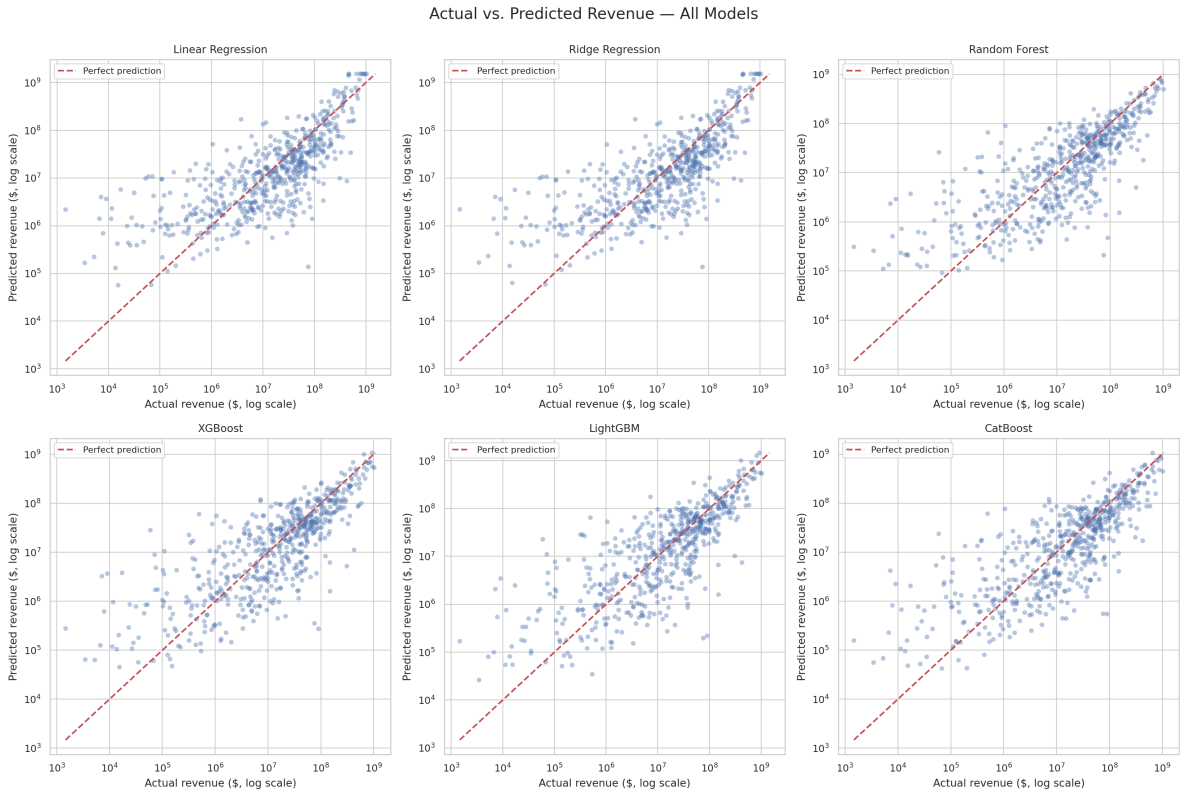


Figure 3: Actual vs predicted revenue, both axes on log scale, one panel per model. Tighter clouds along the diagonal indicate better fit. The linear models (top row) show visible heteroscedasticity and a systematic over-prediction for small-revenue films; the tree ensembles (bottom row) recover the relationship across the full nine-order-of-magnitude range.

4.1 Residual Analysis

Figure 4 shows that CatBoost residuals in log space are approximately Gaussian with mean near zero and a mild negative skew at the low-revenue end. This skew is the source of most of the remaining MAPE: when the model predicts \$2M for a film whose actual revenue is \$50k, the absolute residual in log space is moderate ($\log 40 \approx 3.7$) but the percentage error is 3,900%. The RMSLE and MedAPE columns of Table 2 are not affected by these outliers, which is why we use them as the headline metrics.

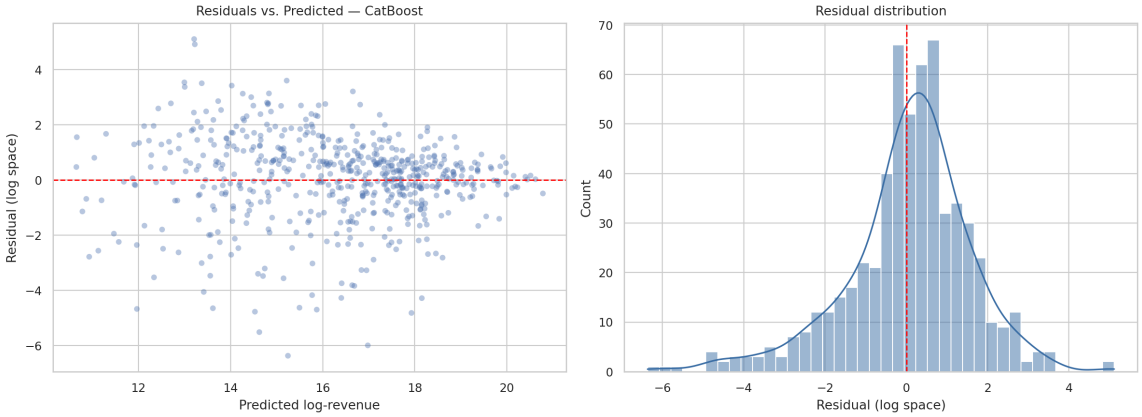


Figure 4: CatBoost residuals in log space. Left: residuals vs predicted log-revenue; right: residual distribution. The residuals are centred near zero and approximately Gaussian, with a heavier left tail corresponding to films whose revenue the model overestimates.

4.2 Feature Importance

Figure 5 ranks the top 20 features by CatBoost's internal importance scores. Five engineered numerical features dominate: `popularity`, `budget_per_minute`, `budget_log`, `release_year`, and `runtime`. The high rank of the engineered `budget_per_minute` (production density) and `budget_log` features above the raw `budget` confirms that the log transform is informative even as an input, not just as a target.

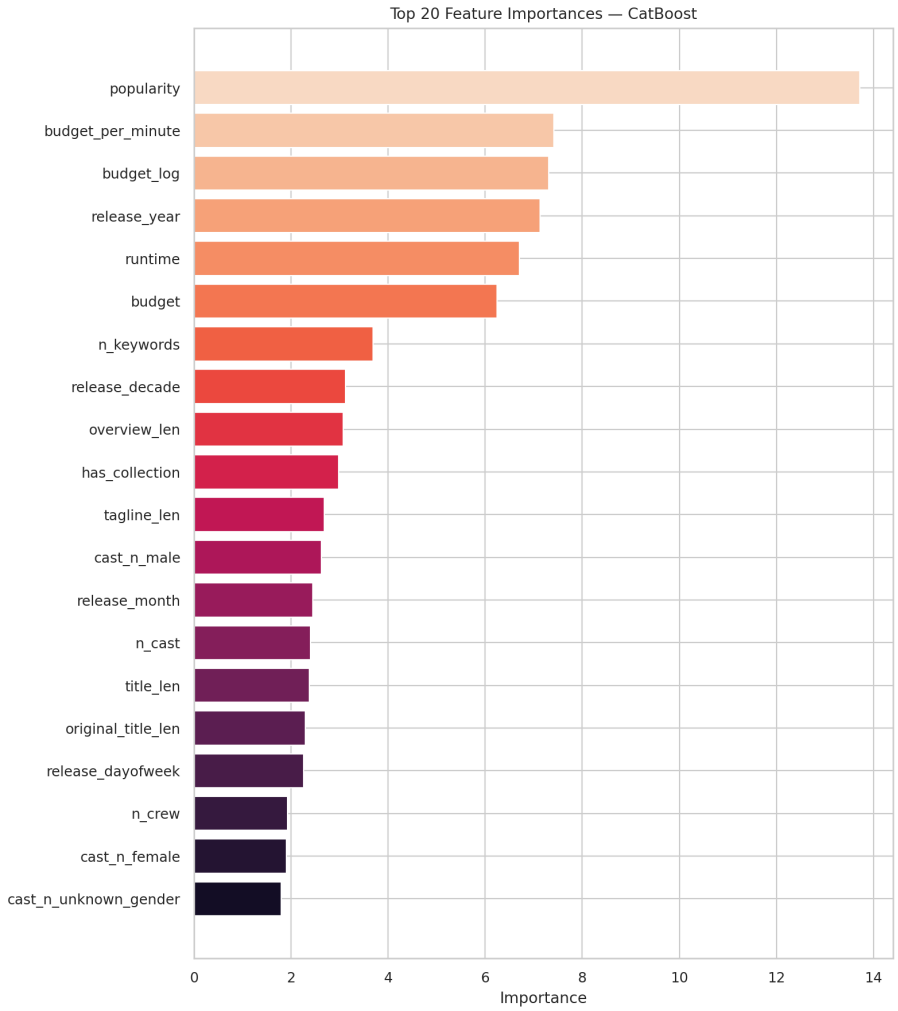


Figure 5: Top 20 features by CatBoost importance. The five most important features are all continuous; engineered budget and temporal features outrank the raw budget column.

4.3 Ablation Study

To attribute performance to feature groups rather than individual features, we re-ran CatBoost 5-fold cross-validation with each of the eight semantic groups in Table 1 *removed* in turn. The reported quantity is $\Delta\text{RMSLE} = \text{RMSLE}_{\text{ablated}} - \text{RMSLE}_{\text{baseline}}$: larger values mean the group is more important. Results are in Figure 6 and Table 3.

Group removed	CV-RMSLE	Δ vs. baseline
Baseline (all features)	1.584	—
Budget	1.717	+0.133
Popularity/runtime	1.693	+0.110
Temporal	1.665	+0.081
Genre flags	1.613	+0.029
Presence flags	1.593	+0.009
Language	1.591	+0.007
Counts	1.585	+0.001
Text lengths	1.568	−0.016

Table 3: Ablation: 5-fold CV-RMSLE when one feature group is dropped. The last row shows that text-length features marginally hurt CV performance.

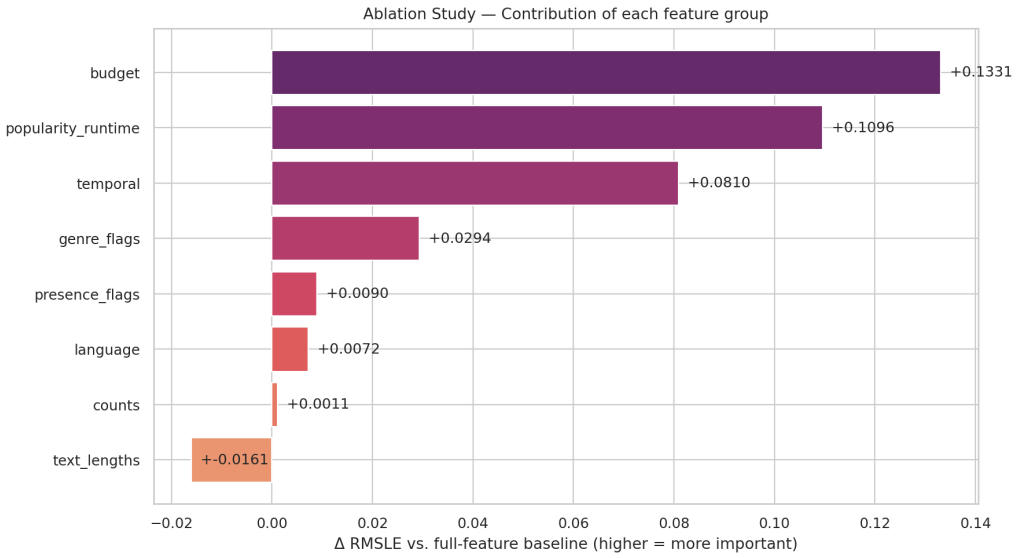


Figure 6: Ablation study: change in 5-fold CV-RMSLE relative to the full-feature baseline when each feature group is removed. Bars to the right are groups whose removal hurts performance; the single bar to the left (`text_lengths`) is a group whose removal actually *improves* performance, suggesting it acts as noise.

The three dominant groups — *budget*, *popularity/runtime*, and *temporal* — together account for +0.32 RMSLE if jointly removed, which is more than the gap between the strongest tree model and the weakest linear model ($1.670 - 1.539 = 0.131$). *Genre flags* contribute a modest but clearly positive +0.029. *Counts* (number of cast/crew/genres/etc.) contribute essentially nothing on top of the dominant signals, and *text lengths* are mildly counter-productive: the CatBoost model trained without them is slightly *better* on cross-validation, which suggests overview/title/tagline lengths act mostly as label-correlated noise for this sample size.

5 Discussion

Comparison with our midterm result. The midterm submission used a single XGBoost model on six raw features and reached $R^2 = 0.6352$ with MAPE $\approx 2{,}096\%$. The reworked pipeline raises hold-out R^2 to 0.709 (Random Forest) and lowers MAPE to 547–612% across the tree ensembles. The largest individual improvement is the log-target transformation, which makes the loss function and the target distribution match each other and is responsible for nearly all of the MAPE reduction. The second-largest improvement is the richer feature set: the ablation in Section 4.3 shows that the three dominant feature groups would, if individually omitted, push RMSLE back well above the linear-model level.

Comparing the six algorithms. The four tree-based ensembles cluster tightly around RMSLE $\approx$ 1.54–1.58 (CV $\approx$ 1.58–1.64). Their differences are within the standard error of cross-validation and we therefore treat them as effectively tied on this task. The linear models lag by about 0.1 RMSLE in absolute terms and produce visibly worse calibration on the scatter plots in Figure 3. We attribute this to two non-linearities the linear models cannot capture: the diminishing-returns relationship between budget and revenue, and the interaction between budget and genre (action films return more revenue per budget dollar than dramas). Among the tree ensembles, CatBoost achieves the lowest CV-RMSLE with the smallest standard deviation, which is consistent with its ordered-boosting mechanism reducing the target leakage that vanilla gradient boosting can suffer on small datasets.

Why MAPE remains high. Even the best model reports a MAPE near 560%, which at first glance looks alarming. The MedAPE of $\approx 65\%$ tells the more honest story: half of the test films are predicted to within $\pm 65\%$ of their true revenue, which is reasonable given that the input features cover at most the production side and contain no information about the marketing campaign, critical reception, competing releases, or release strategy. The remaining mass of the MAPE distribution comes from a small number of films at the bottom of the revenue distribution where any non-trivial dollar-space error becomes a large percentage error by construction. This is not an artefact of the model but of MAPE itself, which is why RMSLE — the metric used by the Kaggle competition leaderboard — is preferred for this task.

Limitations.

- **Sample size.** 3,000 films is modest for the dimensionality we use; the ablation suggests we are already at the noise floor for some feature groups (`counts`, `text_lengths`).
- **Cast/crew identities ignored.** We use only counts and gender mix; learnable embeddings of individual actors and directors would almost certainly add signal but require more samples or external data to estimate reliably.
- **No hyperparameter tuning.** We fixed reasonable defaults for reproducibility; tuning each algorithm with nested cross-validation would likely tighten the gap between the four tree models further without changing the qualitative ranking.
- **Inflation.** The revenue is reported in nominal dollars aggregated across decades; an inflation-adjusted target would change the temporal feature dynamics. We leave this for future work.

6 Conclusion

We presented a clean, reproducible study comparing six regression algorithms for movie revenue prediction on the TMDB Box Office dataset. With a log-target transformation, an engineered 49-feature space, and consistent preprocessing, gradient-boosted ensembles convincingly outperform linear baselines, and CatBoost achieves the best cross-validation RMSLE at 1.584 ± 0.029. An ablation study isolates budget, popularity/runtime, and temporal features as the three dominant contributors and shows that genre flags add a small but real signal, while text-length and metadata-count features are at or below the noise floor for this dataset.

Compared with the midterm baseline of an isolated XGBoost achieving $R^2 = 0.6352$, the present pipeline reaches $R^2 = 0.709$ on the same hold-out split, halves the MAPE, and provides a defensible comparative analysis instead of a single number. The code is released at the project's GitHub repository together with a fixed random seed; running `python main.py` reproduces every number and figure in this report.

References

[1] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD*, 2016.

[2] G. Ke et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *NeurIPS*, 2017.

[3] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. CatBoost: Unbiased Boosting with Categorical Features. In *NeurIPS*, 2018.

[4] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[5] The Movie Database. TMDB Box Office Prediction (Kaggle competition data). https://www.kaggle.com/c/tmdb-box-office-prediction.